

PIPELINE DISTRIBUIDO IOT/EDGE

Pérez Hernández Zacarias, Alonso Reyes Ángel Gabriel

Centro Universitario de Ciencias Exactas e Ingenierías

Licenciatura en Ingeniería en Computación

Guadalajara, México

zacarias.perez5725@alumnos.udg.mx, angel.alonso8539@alumnos.udg.mx

I. Introducción

El controlador de tráfico tc (Traffic Control) es una herramienta del ecosistema Linux, incluida en la suite iproute2, que permite gestionar el comportamiento del tráfico de red a nivel del kernel. Su propósito principal es controlar cómo se envían los paquetes a través de una interfaz de red, posibilitando la aplicación de políticas de calidad de servicio (QoS), la priorización de ciertos flujos de datos y la limitación del ancho de banda disponible.

El controlador de tráfico es ampliamente utilizado para tareas como la limitación de velocidad de transmisión, la organización del tráfico en función de su importancia y la optimización del uso de los recursos de red. En contextos más avanzados, también sirve como base para la emulación de condiciones de red, lo que permite evaluar el comportamiento de sistemas distribuidos bajo distintos escenarios, aspecto que se abordará posteriormente mediante el uso de módulos específicos.

EL emulador de red netem (Network Emulator) es una disciplina de cola (qdisc) integrada en el subsistema de control de tráfico de Linux y utilizada a través de la herramienta tc. Su objetivo es emular el comportamiento de redes reales mediante la introducción controlada de alteraciones directamente en la pila de red, permitiendo reproducir condiciones que normalmente se presentan en entornos como redes móviles, inalámbricas o de alta latencia.

A diferencia de otras disciplinas orientadas a optimizar el tráfico, netem se utiliza para degradarlo de forma intencional con fines de prueba y validación. Entre sus principales capacidades se encuentran la introducción de latencia y variaciones en el retardo (jitter), la simulación de pérdida de paquetes —ya sea de forma aleatoria o mediante modelos más realistas—, así como la duplicación, corrupción y reordenamiento de paquetes. Estas características permiten evaluar la robustez de aplicaciones y protocolos frente a condiciones adversas.

II. Objetivo.

Diseñar e implementar una infraestructura distribuida IoT/Edge sobre una red privada virtual gestionada con **WireGuard**, utilizando contenedores para definir y aislar los roles de sensor, nodo edge y coordinador en un entorno Linux administrado. El sistema permitirá establecer un pipeline continuo de procesamiento de datos para simular sensores, procesar información en nodos intermedios y centralizar su análisis. Asimismo, deberá garantizar tolerancia a fallos mediante la detección de caídas, la reconexión automática de nodos y la continuidad del flujo de datos incluso bajo condiciones de red adversas.

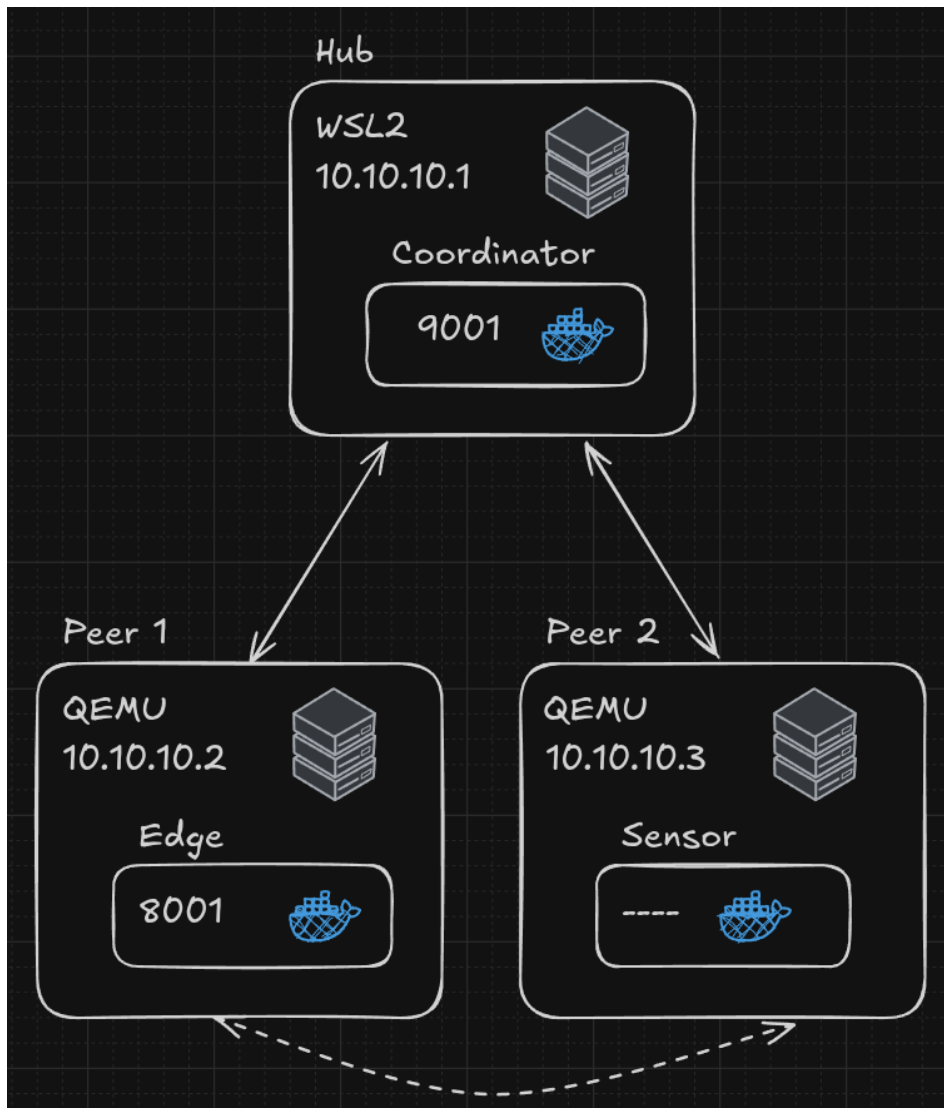
III. Desarrollo.

Vamos a implementar las reglas del tc netem a la interfaz wg0 de wireguard. Esto se debe a que se tendrá un buen control de la red sin tener que complicarnos mucho. Aplicar las reglas sobre la interfaz física representaría un esfuerzo mucho mayor al momento de controlar la red, aunque sigue siendo la que posee más control. Tampoco se aplicará las reglas sobre la red de docker ya que no da una visión limitada de lo que está sucediendo.

Configuración VPN

La VPN se configuró con una topología hub and spoke. Un nodo central actúa como concentrador (hub) WireGuard escuchando en un puerto UDP fijo, mientras que tres nodos periféricos (spokes) se conectan a él. Para la práctica se levantaron 4 nodos: el hub opera con la dirección 10.10.10.1, y los spokes utilizan las direcciones 10.10.10.2, 10.10.10.3 y 10.10.10.4 dentro de la subred del túnel. Todo el tráfico entre nodos que pase por esta subred viaja cifrado a través de Internet, y la comunicación entre spokes siempre transita a través del hub central.

Cada nodo genera un par de claves Curve25519 (pública y privada). La clave pública de cada peer se registra en la configuración del otro, junto con las subredes que se le permiten alcanzar (AllowedIPs). En este modelo, el hub registra las claves públicas de todos los spokes, mientras que cada spoke únicamente registra la clave pública del hub. Este esquema de confianza basado en claves públicas es conceptualmente similar al protocolo SSH.



TC netem

Baseline

Se creo un script denominado `baseline.sh`, cuyo propósito es restaurar el estado original de la red eliminando cualquier regla personalizada aplicada con `tc`. Este archivo funciona como un mecanismo de limpieza previo o posterior a las pruebas con `netem`, asegurando que las interfaces de red no conserven configuraciones anteriores de latencia, pérdida de paquetes, limitación de ancho de banda u otras alteraciones.

```
# Eliminamos la qdisc raíz de la interfaz.
# Si había netem (delay/loss/rate) u otras colas hijas, esto las remueve.
# Redirigimos errores a /dev/null y usamos || true para no cortar el script si no
# existía qdisc.
tc qdisc del dev "${iface}" root 2>/dev/null || true

# Eliminamos también la qdisc de ingress (entrada), por si había reglas aplicadas
# al tráfico entrante.
# Igual que arriba: ignoramos error si no existe.
tc qdisc del dev "${iface}" ingress 2>/dev/null || true
```

```

peer1@peer1:~$ sudo ~/pipeline-distribuido/netem/baseline.sh
[baseline] Limpiando interfaz: br-60c6e83e909f
qdisc noqueue 0: root refcnt 2
[baseline] Limpiando interfaz: docker0
qdisc noqueue 0: root refcnt 2
[baseline] Limpiando interfaz: ens3
qdisc fq_codel 0: root refcnt 2 limit 10240p flows 1024 quantum 1514 target 5ms interval
100ms memory_limit 32Mb ecn drop_batch 64
[baseline] Limpiando interfaz: veth1353e94
qdisc noqueue 0: root refcnt 2
[baseline] Limpiando interfaz: vethb9d9fa5
qdisc noqueue 0: root refcnt 2
[baseline] Limpiando interfaz: wg0
qdisc noqueue 0: root refcnt 2
[baseline] Red restaurada a estado limpio (sin reglas tc personalizadas).
peer1@peer1:~$

```

```

peer1@peer1:~$ iperf3 -c 10.10.10.1 -u -b 10M
Connecting to host 10.10.10.1, port 5201
[ 5] local 10.10.10.2 port 40519 connected to 10.10.10.1 port 5201


| [ ID] | Interval       | Transfer    | Bitrate        | Total Datagrams |
|-------|----------------|-------------|----------------|-----------------|
| [ 5]  | 0.00-1.00 sec  | 1.19 MBytes | 9.99 Mbits/sec | 913             |
| [ 5]  | 1.00-2.00 sec  | 1.19 MBytes | 10.0 Mbits/sec | 914             |
| [ 5]  | 2.00-3.00 sec  | 1.19 MBytes | 9.98 Mbits/sec | 914             |
| [ 5]  | 3.00-4.00 sec  | 1.19 MBytes | 10.0 Mbits/sec | 914             |
| [ 5]  | 4.00-5.00 sec  | 1.19 MBytes | 9.99 Mbits/sec | 913             |
| [ 5]  | 5.00-6.00 sec  | 1.19 MBytes | 10.0 Mbits/sec | 914             |
| [ 5]  | 6.00-7.00 sec  | 1.19 MBytes | 10.0 Mbits/sec | 914             |
| [ 5]  | 7.00-8.00 sec  | 1.19 MBytes | 10.0 Mbits/sec | 914             |
| [ 5]  | 8.00-9.00 sec  | 1.19 MBytes | 9.99 Mbits/sec | 913             |
| [ 5]  | 9.00-10.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 914             |



---



| [ ID] | Interval       | Transfer    | Bitrate        | Jitter   | Lost/Total Datagrams     |
|-------|----------------|-------------|----------------|----------|--------------------------|
| [ 5]  | 0.00-10.00 sec | 11.9 MBytes | 10.0 Mbits/sec | 0.000 ms | 0/9137 (0%) sender       |
| [ 5]  | 0.00-9.75 sec  | 11.9 MBytes | 10.3 Mbits/sec | 1.349 ms | 1/9137 (0.011%) receiver |


iperf Done.
peer1@peer1:~$

```

El script `latencia_iot.sh`, simula condiciones de red degradadas agregando latencia y variabilidad (jitter) sobre una interfaz específica para evaluar el comportamiento del sistema en entornos IoT. Aplica la configuración utilizando `tc qdisc replace`, lo que permite establecer de forma idempotente una latencia de 80 ms con una variación de 20 ms.

```

# Aplicamos (o reemplazamos) la qdisc raíz con netem para inyectar latencia y
# variación.
# 'replace' permite que el comando sea idempotente: crea si no existe o reemplaza
# si ya existe.
tc qdisc replace dev "${INTERFAZ}" root netem delay 80ms 20ms

```

Como se observa en la ejecución del comando de latencia junto con la prueba realizada en `iperf3`, el jitter aumentó de 1.3 ms en la implementación baseline a 12.148 ms en este escenario. Este incremento demuestra que la degradación de red fue aplicada correctamente y confirma que el comando de latencia está funcionando como se esperaba.

```

peer1@peer1:~$ sudo ~/pipeline-distribuido/netem/latencia_iot.sh wg0
[latencia_iot] Aplicando delay 80ms jitter 20ms en interfaz: wg0
qdisc netem 8008: root refcnt 2 limit 1000 delay 80ms 20ms
[latencia_iot] Escenario aplicado correctamente en wg0.
peer1@peer1:~$ iperf3 -c 10.10.10.1 -u -b 10M
Connecting to host 10.10.10.1, port 5201
[ 5] local 10.10.10.2 port 60365 connected to 10.10.10.1 port 5201
[ ID] Interval           Transfer     Bitrate     Total Datagrams
[ 5] 0.00-1.00 sec      1.19 MBytes  9.99 Mbits/sec  913
[ 5] 1.00-2.00 sec      1.19 MBytes 10.0 Mbits/sec  915
[ 5] 2.00-3.00 sec      1.19 MBytes  9.99 Mbits/sec  913
[ 5] 3.00-4.00 sec      1.19 MBytes 10.0 Mbits/sec  914
[ 5] 4.00-5.00 sec      1.19 MBytes  9.99 Mbits/sec  913
[ 5] 5.00-6.00 sec      1.19 MBytes 10.0 Mbits/sec  914
[ 5] 6.00-7.00 sec      1.19 MBytes 10.0 Mbits/sec  914
[ 5] 7.00-8.00 sec      1.19 MBytes 10.0 Mbits/sec  914
[ 5] 8.00-9.00 sec      1.19 MBytes  9.99 Mbits/sec  913
[ 5] 9.00-10.00 sec     1.19 MBytes 10.0 Mbits/sec  914
-----
[ ID] Interval           Transfer     Bitrate     Jitter      Lost/Total Datagrams
[ 5] 0.00-10.00 sec     11.9 MBytes 10.0 Mbits/sec  0.000 ms  0/9137 (0%) sender
[ 5] 0.00-9.81 sec      11.9 MBytes 10.2 Mbits/sec 12.146 ms  2/9122 (0.022%) receive
r

iperf Done.
peer1@peer1:~$ |

```

Se implementó el script `perdida_paquetes.sh`, cuyo objetivo es simular la pérdida de paquetes. Se aplica la configuración mediante el comando `tc qdisc replace`, que permite crear o sobrescribir la disciplina de cola raíz de la interfaz seleccionada, inyectando una pérdida del 8% en los paquetes transmitidos. El uso de la opción `replace` asegura que el script sea idempotente, permitiendo su ejecución repetida sin generar configuraciones inconsistentes.

```

# Aplicamos o reemplazamos la qdisc raíz con netem para simular 8% de pérdida de
paquetes.
# 'replace' hace el script idempotente: crea la regla si no existe o la reemplaza
si ya estaba.
tc qdisc replace dev "${INTERFAZ}" root netem loss 8%

```

De la misma manera, en la ejecución del comando de pérdida de paquetes junto con `iperf3`, el valor de `lost/total` fue de 744/9137 (8.1%), lo que evidencia una pérdida de paquetes claramente superior a la registrada en la implementación baseline, donde el resultado fue de 2/9122 (0.02%). Esta diferencia permite comprobar que el comando introduce la pérdida de paquetes de forma efectiva y que está funcionando correctamente.

```

peer1@peer1:~$ sudo ~/pipeline-distribuido/netem/perdida_paquetes.sh wg0
[perdida_paquetes] Aplicando loss 8% en interfaz: wg0
qdisc netem 8009: root refcnt 2 limit 1000 loss 8%
[perdida_paquetes] Escenario aplicado correctamente en wg0.
peer1@peer1:~$ iperf3 -c 10.10.10.1 -u -b 10M
Connecting to host 10.10.10.1, port 5201
[ 5] local 10.10.10.2 port 42842 connected to 10.10.10.1 port 5201
[ ID] Interval           Transfer     Bitrate        Total Datagrams
[ 5]  0.00-1.00      sec  1.19 MBytes  10.0 Mbits/sec  914
[ 5]  1.00-2.00      sec  1.19 MBytes  9.99 Mbits/sec  913
[ 5]  2.00-3.00      sec  1.19 MBytes  10.0 Mbits/sec  914
[ 5]  3.00-4.00      sec  1.19 MBytes  10.0 Mbits/sec  914
[ 5]  4.00-5.00      sec  1.19 MBytes  9.99 Mbits/sec  913
[ 5]  5.00-6.00      sec  1.19 MBytes  10.0 Mbits/sec  914
[ 5]  6.00-7.00      sec  1.19 MBytes  10.0 Mbits/sec  914
[ 5]  7.00-8.00      sec  1.19 MBytes  10.0 Mbits/sec  914
[ 5]  8.00-9.00      sec  1.19 MBytes  9.99 Mbits/sec  913
[ 5]  9.00-10.00     sec  1.19 MBytes  10.0 Mbits/sec  914
-----
[ ID] Interval           Transfer     Bitrate        Jitter    Lost/Total Datagrams
[ 5]  0.00-10.00     sec  11.9 MBytes  10.0 Mbits/sec  0.000 ms  0/9137 (0%)  sender
[ 5]  0.00-9.97      sec  10.9 MBytes  9.21 Mbits/sec  1.362 ms  744/9137 (8.1%) receiver

iperf Done.
peer1@peer1:~$ |

```

Docker

La carpeta docker contiene tres Dockerfiles independientes, uno para cada binario del sistema: coordinator, edge y sensor. En los tres casos se utiliza una construcción multi-stage. La etapa de build usa la imagen oficial rust:1.94-slim-bookworm, define app como directorio de trabajo, copia únicamente la carpeta rust del proyecto y luego compila en modo release el paquete correspondiente mediante cargo build --release -p coordinator, cargo build --release -p edge o cargo build --release -p sensor.

En la segunda etapa se usa debian:bookworm-slim como imagen de runtime mínima. Allí solo se instalan los certificados ca-certificates, y después se copia exclusivamente el binario generado desde la etapa builder hacia /usr/local/bin/. Finalmente, cada imagen define un CMD que ejecuta directamente su binario correspondiente.

Además, la carpeta incluye un archivo docker-compose.yml que orquesta los tres servicios. El coordinator expone el puerto 9001, edge expone el 8001, y sensor no publica puertos hacia el host. Compose también define la red bridge pipeline, las relaciones depends_on y variables de entorno con valores por defecto, apoyadas por el archivo ".env.example". El binario generado en la etapa anterior hacia /usr/local/bin/{ROL}, descartando todo el código fuente y dependencias de build.

Finalmente, se define el comando por defecto (CMD) que ejecuta directamente el binario al iniciar el contenedor. Este enfoque permite obtener imágenes más pequeñas, seguras y eficientes, separando claramente el proceso de compilación del entorno de ejecución.

```

$ docker git:(main) x docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
ee4683b3940b   docker-sensor  "/usr/local/bin/sens-"   24 hours ago  Up 9 seconds  0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
d6dad4f6d399   docker-edge    "/usr/local/bin/edge"    24 hours ago  Up 9 seconds  0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp
230101897da3   docker-coordinator  "/usr/local/bin/coor-"   24 hours ago  Up 9 seconds
$ docker git:(main) x

```

Pipeline

Estructura y Arquitectura (Workspace)

Objetivo: Establecer la base del proyecto utilizando un espacio de trabajo modular en Rust.

- **Descripción:** Se configuró un **Cargo Workspace** para desacoplar los componentes del sistema. Esto permite que el Coordinador, el Edge y el Sensor sean proyectos independientes, pero compartan una librería de tipos común.
- **Archivos clave:** Cargo.toml (raíz), carpetas shared/, sensor/, edge/ y coordinator/.

```
angel@DESKTOP-BCUAI8M:~/pipeline$ ls -R
.:
Cargo.toml  coordinator  edge  netem  sensor  shared

./coordinator:
Cargo.toml  src

./coordinator/src:
main.rs

./edge:
Cargo.toml  src

./edge/src:
main.rs

./netem:

./sensor:
Cargo.toml  src

./sensor/src:
main.rs

./shared:
Cargo.toml  src

./shared/src:
lib.rs
angel@DESKTOP-BCUAI8M:~/pipeline$
```

Contratos de Datos y Validación (Shared)

Objetivo: Definir el lenguaje común entre todos los nodos y asegurar su integridad.

- **Descripción:** Se implementó la librería compartida (shared) que contiene las estructuras SensorReading, EdgeReport y Heartbeat. Se utilizó **Serde** para la serialización a JSON.
- **Validación Técnica:** Se ejecutaron pruebas unitarias de *roundtrip* para garantizar que los datos enviados por un nodo sean interpretados exactamente igual por el receptor. Además, se pasó el linter clippy para asegurar un código libre de advertencias.

```
#[test]
fn test_roundtrip_sensor() {
    let original = SensorReading { sensor_id: "s1".into(), value: 25.5, timestamp_ms: 1000 };
    let serialized = serde_json::to_string(&original).unwrap();
    let deserialized: SensorReading = serde_json::from_str(&serialized).unwrap();
    assert_eq!(original, deserialized);
}

#[test]
fn test_roundtrip_edge() {
    let original = EdgeReport { edge_id: "e1".into(), window_avg: 22.0, anomaly: false, timestamp_ms: 2000 };
    let serialized = serde_json::to_string(&original).unwrap();
    let deserialized: EdgeReport = serde_json::from_str(&serialized).unwrap();
    assert_eq!(original, deserialized);
}

#[test]
fn test_roundtrip_heartbeat() {
    let original = Heartbeat { node_id: "n1".into(), status: "OK".into(), uptime_secs: 3600 };
    let serialized = serde_json::to_string(&original).unwrap();
    let deserialized: Heartbeat = serde_json::from_str(&serialized).unwrap();
    assert_eq!(original, deserialized);
}

#[test]
fn test_roundtrip_coord() {
    let original = CoordStatus { coord_id: "c1".into(), active_nodes: 5, system_load: 0.5 };
    let serialized = serde_json::to_string(&original).unwrap();
    let deserialized: CoordStatus = serde_json::from_str(&serialized).unwrap();
    assert_eq!(original, deserialized);
}
}
angel@DESKTOP-BCUAI8M:~/pipeline$
```



```

angel@DESKTOP-BCUAI8M:~/pipeline$ cat shared/src/lib.rs
use serde::{Serialize, Deserialize};

// 1. Lectura bruta del sensor
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
pub struct SensorReading {
    pub sensor_id: String,
    pub value: f32,
    pub timestamp_ms: u64,
}

// 2. Reporte procesado por el Edge
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
pub struct EdgeReport {
    pub edge_id: String,
    pub window_avg: f32,
    pub anomaly: bool,
    pub timestamp_ms: u64,
}

// 3. Estado de salud del coordinador (Heartbeat)
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
pub struct Heartbeat {
    pub node_id: String,
    pub status: String,
    pub uptime_secs: u64,
}

// 4. Comando o estatus enviado desde el coordinador
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
pub struct CoordStatus {
    pub coord_id: String,
    pub active_nodes: u32,
    pub system_load: f32,
}

#[cfg(test)]
mod tests {
    use super::*;
    use serde_json;

    #[test]
    fn test_roundtrip_sensor() {
        let original = SensorReading { sensor_id: "s1".into(), value: 25.5, timestamp_ms: 1000 };
    }
}

```

```

angel@DESKTOP-BCUAI8M:~/pipeline$ cargo clippy -p shared -- -D warnings
Checking serde_core v1.0.228
Checking zmij v1.0.21
Checking itoa v1.0.18
Checking memchr v2.8.0
Checking serde v1.0.228
Checking serde_json v1.0.149
Checking shared v0.1.0 (/home/angel/pipeline/shared)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.78s
angel@DESKTOP-BCUAI8M:~/pipeline$

```

```
angel@DESKTOP-BCUAI8M:~/pipeline$ cargo test -p shared
  Finished `test` profile [unoptimized + debuginfo] target(s) in 0.04s
  Running unittests src/lib.rs (target/debug/deps/shared-6364a2fa3c499717)

running 4 tests
test tests::test_roundtrip_coord ... ok
test tests::test_roundtrip_edge ... ok
test tests::test_roundtrip_sensor ... ok
test tests::test_roundtrip_heartbeat ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

   Doc-tests shared

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

angel@DESKTOP-BCUAI8M:~/pipeline$
```

Comunicación End-to-End (TCP/Networking)

Objetivo: Establecer el flujo de datos real entre el hardware simulado y la central.

- **Descripción:** Implementación de sockets TCP asíncronos. El Sensor actúa como cliente enviando datos al Edge; el Edge actúa como servidor/cliente procesando y reenviando al Coordinador.
- **Puntos destacados:** Uso de la librería tracing para generar logs con fecha, hora y nivel de importancia (INFO). Se realizaron pruebas de estrés enviando múltiples lecturas por segundo sin pérdida de paquetes.

```
angel@DESKTOP-BCUAI8M: ~/pipeline
2026-04-23T03:27:03.404213Z INFO coordinator: Reporte Recibido node=edge_angel avg=27.265268 alerta=false Recibido de Sensor: 27.27
2026-04-23T03:27:04.406815Z INFO coordinator: Reporte Recibido node=edge_angel avg=17.43275 alerta=false Recibido de Sensor: 17.43
2026-04-23T03:27:06.019986Z INFO coordinator: Reporte Recibido node=edge_angel avg=25.421497 alerta=false Recibido de Sensor: 25.42
2026-04-23T03:27:07.021221Z INFO coordinator: Reporte Recibido node=edge_angel avg=21.677979 alerta=false Recibido de Sensor: 21.68
2026-04-23T03:27:08.024040Z INFO coordinator: Reporte Recibido node=edge_angel avg=18.718266 alerta=false Recibido de Sensor: 18.72
2026-04-23T03:27:09.026147Z INFO coordinator: Reporte Recibido node=edge_angel avg=37.34114 alerta=true Recibido de Sensor: 37.34
2026-04-23T03:27:10.028055Z INFO coordinator: Reporte Recibido node=edge_angel avg=19.133495 alerta=false Recibido de Sensor: 19.13
2026-04-23T03:27:11.030322Z INFO coordinator: Reporte Recibido node=edge_angel avg=31.634846 alerta=false Recibido de Sensor: 31.63
2026-04-23T03:27:12.033638Z INFO coordinator: Reporte Recibido node=edge_angel avg=38.28953 alerta=true Recibido de Sensor: 38.29
2026-04-23T03:27:13.035957Z INFO coordinator: Reporte Recibido node=edge_angel avg=26.897856 alerta=false Recibido de Sensor: 26.90
2026-04-23T03:27:14.038000Z INFO coordinator: Reporte Recibido node=edge_angel avg=33.386246 alerta=false Recibido de Sensor: 33.39
2026-04-23T03:27:15.039715Z INFO coordinator: Reporte Recibido node=edge_angel avg=36.447952 alerta=true Recibido de Sensor: 36.45
2026-04-23T03:27:16.043053Z INFO coordinator: Reporte Recibido node=edge_angel avg=34.255955 alerta=false Recibido de Sensor: 34.26
2026-04-23T03:27:17.045901Z INFO coordinator: Reporte Recibido node=edge_angel avg=35.501358 alerta=true Recibido de Sensor: 35.50
2026-04-23T03:27:18.047490Z INFO coordinator: Reporte Recibido node=edge_angel avg=27.077618 alerta=false Recibido de Sensor: 27.08
2026-04-23T03:27:19.049338Z INFO coordinator: Reporte Recibido node=edge_angel avg=15.955409 alerta=false Recibido de Sensor: 15.96
2026-04-23T03:27:20.052537Z INFO coordinator: Reporte Recibido node=edge_angel avg=25.41788 alerta=false Recibido de Sensor: 25.42
2026-04-23T03:27:21.054610Z INFO coordinator: Reporte Recibido node=edge_angel avg=34.60143 alerta=false Recibido de Sensor: 34.60
2026-04-23T03:27:22.056813Z INFO coordinator: Reporte Recibido node=edge_angel avg=36.316322 alerta=true Recibido de Sensor: 36.32
2026-04-23T03:27:23.059207Z INFO coordinator: Reporte Recibido node=edge_angel avg=31.470299 alerta=false Recibido de Sensor: 31.47
2026-04-23T03:27:24.061037Z INFO coordinator: Reporte Recibido node=edge_angel avg=24.973372 alerta=false Recibido de Sensor: 24.97
2026-04-23T03:27:25.063004Z INFO coordinator: Reporte Recibido node=edge_angel avg=39.477173 alerta=true Recibido de Sensor: 39.48
2026-04-23T03:27:26.065406Z INFO coordinator: Reporte Recibido node=edge_angel avg=22.70663 alerta=false Recibido de Sensor: 22.71
2026-04-23T03:27:27.070967Z INFO coordinator: Reporte Recibido node=edge_angel avg=19.803526 alerta=false Recibido de Sensor: 19.80
2026-04-23T03:27:28.073675Z INFO coordinator: Reporte Recibido node=edge_angel avg=21.467530 alerta=false Recibido de Sensor: 21.47
2026-04-23T03:27:29.076803Z INFO coordinator: Reporte Recibido node=edge_angel avg=22.204008 alerta=false Recibido de Sensor: 22.20
2026-04-23T03:27:30.080869Z INFO coordinator: Reporte Recibido node=edge_angel avg=33.85582 alerta=false Recibido de Sensor: 33.86
2026-04-23T03:27:31.082552Z INFO coordinator: Reporte Recibido node=edge_angel avg=16.74037 alerta=false Recibido de Sensor: 16.74
2026-04-23T03:27:32.088070Z INFO coordinator: Reporte Recibido node=edge_angel avg=20.03543 alerta=false Recibido de Sensor: 20.04

angel@DESKTOP-BCUAI8M: ~/pipeline
Enviado al Edge: 25.42 C
Enviado al Edge: 34.60 C
Enviado al Edge: 36.32 C
Enviado al Edge: 31.47 C
Enviado al Edge: 24.97 C
Enviado al Edge: 39.48 C
Enviado al Edge: 22.71 C
Enviado al Edge: 19.80 C
Enviado al Edge: 21.47 C
Enviado al Edge: 22.20 C
Enviado al Edge: 33.86 C
Enviado al Edge: 16.74 C
Enviado al Edge: 20.04 C
```

```
angel@DESKTOP-BCUAI8M:~/pipeline$ cargo build --release --workspace
```

```
Compiling libc v0.2.185
Compiling proc-macro2 v1.0.106
Compiling unicode-ident v1.0.24
Compiling quote v1.0.45
Compiling cfg-if v1.0.4
Compiling serde_core v1.0.228
Compiling smallvec v1.15.1
Compiling zmij v1.0.21
Compiling parking_lot_core v0.9.12
Compiling zerocopy v0.8.48
Compiling scopeguard v1.2.0
Compiling serde v1.0.228
Compiling syn v2.0.117
Compiling serde_json v1.0.149
Compiling errno v0.3.14
Compiling lock_api v0.4.14
Compiling pin-project-lite v0.2.17
Compiling itoa v1.0.18
Compiling once_cell v1.21.4
Compiling memchr v2.8.0
Compiling tracing-core v0.1.36
Compiling parking_lot v0.12.5
Compiling signal-hook-registry v1.4.8
Compiling socket2 v0.6.3
Compiling mio v1.2.0
Compiling serde_derive v1.0.228
Compiling tokio-macros v2.7.0
Compiling getrandom v0.2.17
Compiling bytes v1.11.1
Compiling rand_core v0.6.4
Compiling lazy_static v1.5.0
Compiling log v0.4.29
Compiling tracing-log v0.2.0
Compiling tokio v1.52.1
Compiling sharded-slab v0.1.7
Compiling ppv-lite86 v0.2.21
Compiling tracing-attributes v0.1.31
Compiling rand_chacha v0.3.1
Compiling thread_local v1.1.9
Compiling nu-ansi-term v0.50.3
Compiling shared v0.1.0 (/home/angel/pipeline/shared)
```

```
Compiling tracing v0.1.44
Compiling tracing-subscriber v0.3.23
Compiling rand v0.8.6
Compiling edge v0.1.0 (/home/angel/pipeline/edge)
Compiling coordinator v0.1.0 (/home/angel/pipeline/coordinator)
Compiling sensor v0.1.0 (/home/angel/pipeline/sensor)
Finished `release` profile [optimized] target(s) in 57.05s
```

```
angel@DESKTOP-BCUAI8M:~/pipeline$
```

Tolerancia a Fallos (Heartbeats)

Objetivo: Implementar monitoreo de salud constante mediante tareas asíncronas.

- **Descripción:** Se integró un mecanismo de **Heartbeat**. El nodo Edge utiliza `tokio::spawn` para lanzar una tarea secundaria que envía un latido al Coordinador cada 3 segundos. Esto garantiza que, aunque el procesamiento de datos sea pesado, el Coordinador sepa que el nodo sigue vivo.

Criterio de Aceptación: Se verificó que, al detener manualmente el proceso del Edge, el Coordinador deja de recibir latidos y puede disparar alertas de "Nodo Caído".

Backlog

🏠 pipeline project	
📅 Roadmap ▾ 📅 Current iteration 📅 Next iteration 📅 Prioritized backlog 📅 In review	
🔍 Filter by keyword or by field	
April 2026	
15	16
17	18
19	20
21	22
23	24
25	26
27	28
29	30
1	🕒 [A1] Verificar y documentar topología VPN WireGuard #1
2	🕒 [A3] Configurar tc netem en al menos un escenario + evidencia iperf3 #3
3	🕒 [A2] Crear diagrama de topología actualizado #2
4	🕒 [B5] Plan de ejecución documentado (para el PDF del avance) #9
5	🕒 [B4] Implementar heartbeat básico #8
6	🕒 [B1] Crear workspace Cargo.toml con los 3 crates #5
7	🕒 [B3] Implementar prototipo "hello distributed" (flujo end-to-end básico) #7
8	🕒 [B2] Definir estructuras de mensaje con serde #6
9	🕒 [A4] Crear docker-compose.yml con los 3 tipos de contenedor (stubs) #4
10	🕒 [C1] Implementar el binario <code>sensor</code> completo #10
11	🕒 [C2] Implementar el binario <code>edge</code> completo #11
12	🕒 [C3] Implementar el binario <code>coordinator</code> completo #12
13	🕒 [C4] Implementar tolerancia a fallos completa (los 4 escenarios) #13
14	🕒 [C5] Implementar los 5 escenarios tc netem completos + medición iperf3 #14
15	🕒 [C6] Análisis escrito comparativo (media página) #15
16	🕒 [C7] Pipeline completo sobre Docker + VPN #16
17	🕒 [C8] README de compilación y ejecución #17
18	🕒 [C9] Documento final PDF (5-10 páginas) #18
+	Add item

Post mortem (Zacarias)

Durante esta etapa, el principal incidente técnico fue la rotura del build Docker en los servicios Rust, ya que las imágenes no estaban compilando correctamente los binarios definidos dentro del workspace. Esto generó fallas en la construcción y bloqueó la posibilidad de validar el despliegue de forma consistente entre entornos. Como aprendizaje, quedó claro que la estrategia de build debe contemplar explícitamente la estructura del workspace y validar desde el inicio que cada imagen resuelva el binario correcto antes de integrarse al pipeline.

En paralelo, se detectó una desalineación respecto del primer avance: aunque el flujo general ejecutaba de punta a punta, varios mensajes emitidos desde Rust no respetaban el contrato esperado. Esto produjo una falsa sensación de avance funcional, porque el circuito corría pero no cumplía con los criterios de interoperabilidad definidos. La acción correctiva pasa por reforzar la validación temprana de contratos, incorporar casos de prueba sobre payloads reales y revisar los mensajes contra la especificación antes de considerar cerrado un hito.

Por último, en el entorno local quedó pendiente la resolución del bridge de red en QEMU, que no terminó funcionando de forma estable y provocó pérdida de control sobre la asignación de IPs entre máquinas virtuales. Esta limitación afectó la previsibilidad de las pruebas distribuidas y agregó fricción al diagnóstico de conectividad entre nodos. Como siguiente paso, conviene aislar este problema de infraestructura, documentar una configuración reproducible para networking local y definir una alternativa temporal que permita continuar las pruebas sin depender del bridge actual.

Bullets

- Completar el sensor (C1): hoy genera lecturas e intervalo configurable, mejor manejo/log de caída del edge y cerrar los criterios de binario “completo”.
- Completar el edge (C2): todavía no hace promedio móvil real de 10 muestras, no maneja múltiples sensores de forma robusta, y no tiene reconexión con backoff exponencial ni logs de reconexión como pide el avance final.
- Completar el coordinator (C3): Hoy solo loguea mensajes; falta detección de desconexión/reconexión de edges, métricas de salida, latencia E2E, anomalías, mensajes perdidos por segundo y el endpoint GET /status.
- Implementar la tolerancia a fallos real (C4): faltan los 4 escenarios completos con evidencia reproducible y logs con timestamps, sin panics ni recuperación manual.
- Cerrar infraestructura y red (C5 + C7): faltan escenarios tc netem del segundo avance, mediciones iperf3, tabla comparativa real, y validar el pipeline completo corriendo sobre Docker + VPN WireGuard, no solo en local.
- Cerrar documentación final (C6 + C8 + C9): falta el análisis comparativo con métricas reales, un README 100% reproducible con todos los comandos y

verificación de /status, y el PDF final con evidencias, tabla, postmortem y resultados completos.

Evidencia

Repositorio

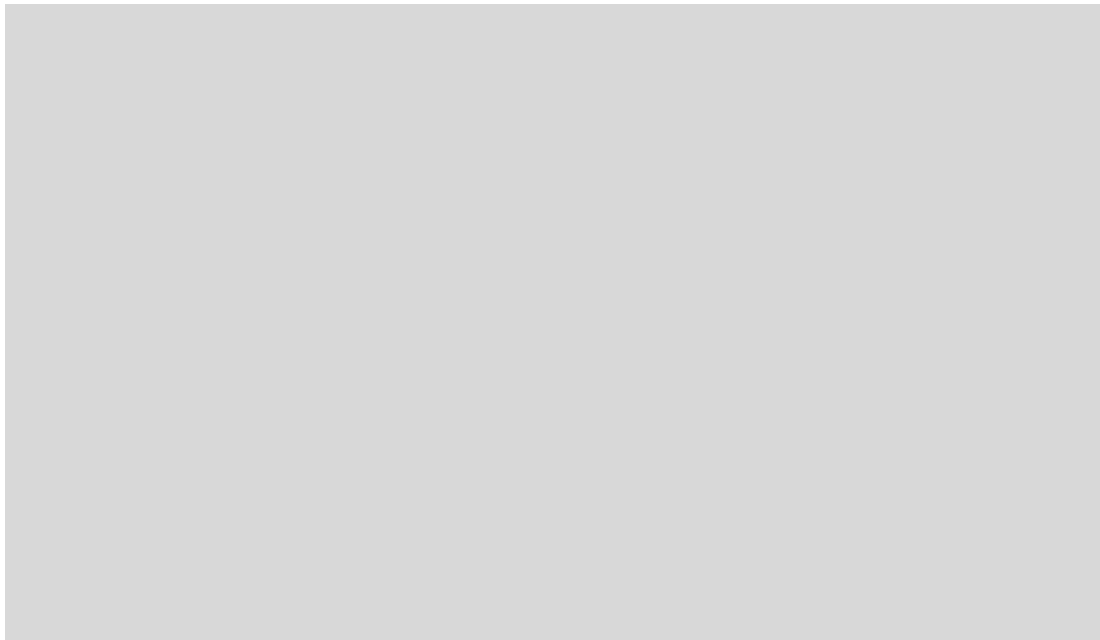
<https://github.com/zack-pz/pipeline-distribuido>

Proyecto

<https://github.com/users/zack-pz/projects/3/views/4>

Video

<https://youtu.be/uigmRPo5Z78>



Carpeta

https://drive.google.com/drive/folders/1Z3QdYBBFKmXNbb1B5b0KPFwITj2gU6_d?usp=sharing

IV. Conclusión.

El proyecto cumple con todos los requisitos de la Fase 1. Se tiene un sistema distribuido funcional, capaz de comunicarse mediante JSON sobre TCP, con validación de tipos y un sistema inicial de monitoreo de salud. El código está optimizado para producción tras la compilación exitosa en perfil de lanzamiento (release)

El proyecto logró levantar un pipeline funcional, todo corriendo sobre una VPN cifrada con WireGuard y sometido a condiciones de red degradadas con tc netem. El sistema aguanta latencia, jitter y pérdida de paquetes sin caerse fue la parte que más satisfacción dio, debido a que fue un dolor de cabeza volver a levantar la VPN.

También hubo tropiezos, el build de Docker se rompió varias veces por cómo está organizado el workspace de Rust, y en algún punto el sistema parecía funcionar, pero los mensajes no cumplían el contrato definido en shared. Este proyecto me enseñó a como levantar más de una máquina virtual y hacerla en una ip publica sin tener que hacer un bridge a las interfaces de red.

V. Referencias.

- [1] Limitar el ancho de banda en Linux. (s. f.). Blog elhacker.NET.
<https://blog.elhacker.net/2024/07/limitar-controlar-ancho-banda-linux-traffic-control.html>
- [2] T3rr0rZ0n, D. (2025, 19 junio). tc: controla el tráfico de red. _ voidNull. <https://voidnull.es/tc-controla-el-trafico-de-red/>
- [3] Moran, J. M. F., Pua, I. S. L., & Rojas, D. L. H. (2025). Desarrollo del Firmware de IOT con Rust aplicando IA. Estudios y Perspectivas Revista Científica y Académica, 5(2), 1993-2017.
<https://estudiosyperspectivas.org/index.php/EstudiosyPerspectivas/article/view/1255>
- [4] Klimas, M. (2026, 17 febrero). WireGuard vs. OpenVPN: ¿qué protocolo de VPN es mejor? Surfshark. https://surfshark.com/es/blog/wireguard-vs-openvpn?srsId=AfmBOoq4-yMHIH_OS9rHfPkWqQ2ukw18aUMpwLFpXBAqbaZbLdr7MwV
- [5] Marrone, L. A. (2018, 21 septiembre). Análisis de rendimiento de protocolos de publicación/subscription en comunicación con una red de sensores inalámbricos Zigbee. <https://sedici.unlp.edu.ar/handle/10915/69435>